

Semantic Understanding and Command Compliance

Semantic Understanding and Command Compliance

1. Course Content
2. Start the Agent
3. Running the Example
 - 3.1 Starting the Program
 - 3.2 Test Cases
 - 3.2.1 Case 1
 - 3.2.2 Case 2
4. Code Analysis

1. Course Content

Run the example program to interact with the robot solely through terminal text; there is no voice input or voice response.

2. Start the Agent

Note: If it's already started, you don't need to start it again.

Enter the command on the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
subscriber created | client_key: 0x1D229138, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1767693398.286449] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1767693398.290468] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x007(2), participant_
id: 0x000(1)
[1767693398.294835] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x1D229138, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1767693398.299603] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
[1767693398.303485] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x008(2), participant_
id: 0x000(1)
[1767693398.330727] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x1D229138, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1767693398.335405] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
```

3. Running the Example

3.1 Starting the Program

For Raspberry Pi 5 users, if the ROS container is not running, you need to start it first.

```
bringup_m3
```

Open the terminal inside the container.

```
exec_m3
```

- Open the terminal on the vehicle's terminal and enter the command:

```
ros2 launch m3_bringup llm_agent_control.launch.py text_chat_mode:=True
```

- Start the text-based interactive terminal. You can start it on either the vehicle's infotainment system or a virtual machine. **Choose one;** you don't need to start it repeatedly on both the virtual machine and the vehicle's infotainment system:

```
ros2 run m3_common_demo text_chat
```

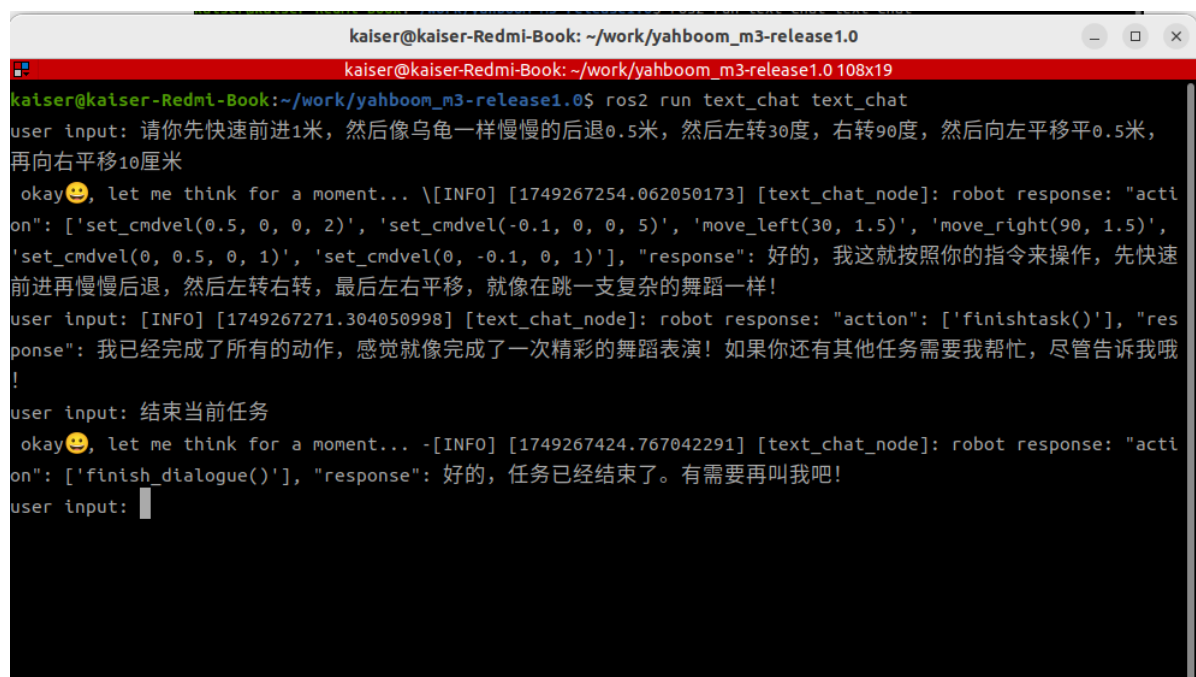
3.2 Test Cases

Here are some sample test cases. Users can create their own dialogue commands.

- Please move forward quickly 1 meter, then slowly retreat 0.5 meters like a tortoise, then turn 30 degrees to the left, 90 degrees to the right, then move horizontally 0.5 meters to the left, and then horizontally 10 centimeters to the right.
- Please perform a dance first, then tell me a joke about cats and dogs.

3.2.1 Case 1

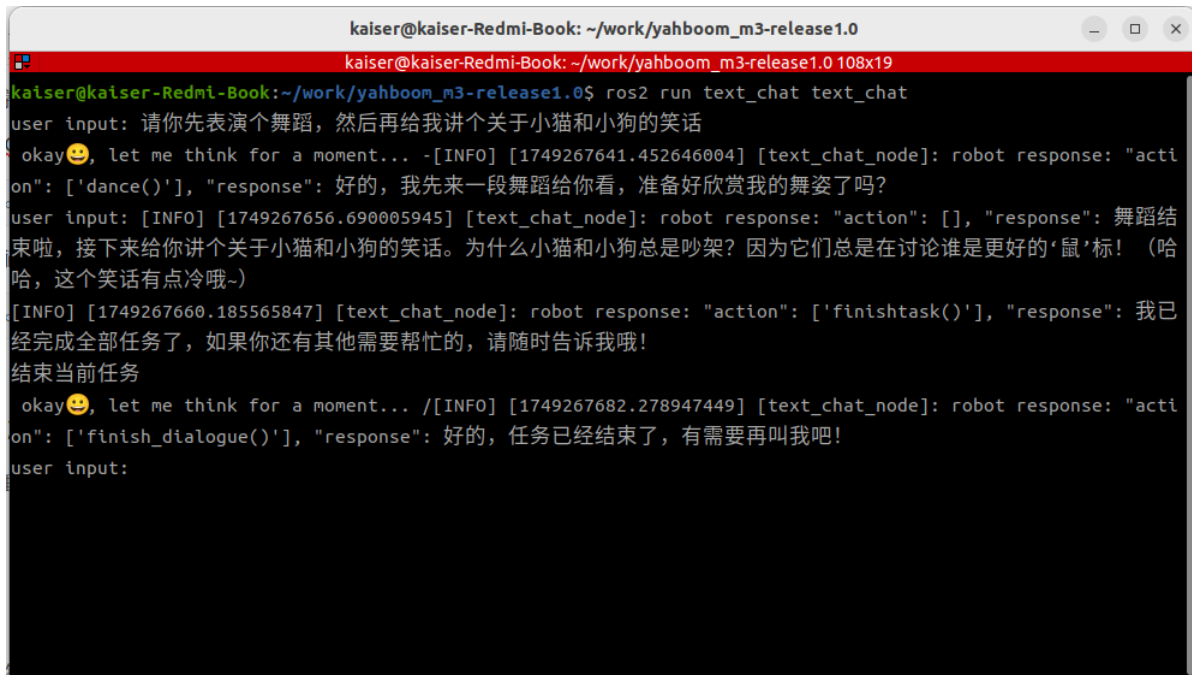
Open a terminal in the virtual machine, enter the test case in the terminal, wait for the model to think, and the AI agent will reply



```
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0 108x19
kaiser@kaiser-Redmi-Book:~/work/yahboom_m3-release1.0$ ros2 run text_chat text_chat
user input: 请你先快速前进1米，然后像乌龟一样慢慢的后退0.5米，然后左转30度，右转90度，然后向左平移0.5米，再向右平移10厘米
okay😊, let me think for a moment... \[INFO] [1749267254.062050173] [text_chat_node]: robot response: "action": ['set_cmdvel(0.5, 0, 0, 2)', 'set_cmdvel(-0.1, 0, 0, 5)', 'move_left(30, 1.5)', 'move_right(90, 1.5)', 'set_cmdvel(0, 0.5, 0, 1)', 'set_cmdvel(0, -0.1, 0, 1)'], "response": "好的，我就按照你的指令来操作，先快速前进再慢慢后退，然后左转右转，最后左右平移，就像在跳一支复杂的舞蹈一样！"
user input: [INFO] [1749267271.304050998] [text_chat_node]: robot response: "action": ['finishtask()'], "response": "我已经完成了所有的动作，感觉就像完成了一次精彩的舞蹈表演！如果你还有其他任务需要我帮忙，尽管告诉我哦！"
user input: 结束当前任务
okay😊, let me think for a moment... -[INFO] [1749267424.767042291] [text_chat_node]: robot response: "action": ['finish_dialogue()'], "response": "好的，任务已经结束了。有需要再叫我吧！"
user input: 
```

3.2.2 Case 2

Similar to Case 1, enter Case 2 in the terminal, and the model will respond and execute the action according to the command.

A terminal window titled 'kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0' with a red title bar. The terminal shows a ROS2 command 'ros2 run text_chat text_chat' being executed. The chatbot responds to a user input about dancing and telling jokes, performing a 'dance()' action and then a 'finishtask()' action. The terminal text is as follows:

```
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0$ ros2 run text_chat text_chat
user input: 请你先表演个舞蹈，然后再给我讲个关于小猫和小狗的笑话
okay😊, let me think for a moment... -[INFO] [1749267641.452646004] [text_chat_node]: robot response: "action": ['dance()'], "response": 好的，我先来一段舞蹈给你看，准备好欣赏我的舞姿了吗？
user input: [INFO] [1749267656.690005945] [text_chat_node]: robot response: "action": [], "response": 舞蹈结束啦，接下来给你讲个关于小猫和小狗的笑话。为什么小猫和小狗总是吵架？因为它们总是在讨论谁是更好的‘鼠’标！（哈哈，这个笑话有点冷哦~）
[INFO] [1749267660.185565847] [text_chat_node]: robot response: "action": ['finishtask()'], "response": 我已经完成全部任务了，如果你还有其他需要帮忙的，请随时告诉我！
结束当前任务
okay😊, let me think for a moment... /[INFO] [1749267682.278947449] [text_chat_node]: robot response: "action": ['finish_dialogue()'], "response": 好的，任务已经结束了，有需要再叫我吧！
user input:
```

4. Code Analysis

This lesson applies the basic program framework of AI embodied intelligence gameplay. For code analysis, refer to the chapter [4.Embodied intelligent functions core source code]